

PROJECT PORTFOLIO

HRD STRATEGIC CIRCLE

Private Knowledge Portal

한국경제인협회 국제경영원 HRD 담당자 네트워크를 위한

초청형 비공개 자료 포털 구축

기획 · 설계 · 개발 · 배포 전 과정 단독 수행

임채헌

기획 · 풀스택 개발 · 배포 운영

2026. 08. 14

총 개발 기간 1일 · 서비스 운영 중

한 장으로 보는 프로젝트

한국경제인협회 국제경영원이 운영하는 **중견기업 이상 HRD 담당자 초청 네트워크**를 위해, 회원 전용 자료 포털을 **기획부터 배포까지 하루 만에 단독으로 완성**했습니다. 커뮤니티가 아닌 **지식 아카이브**로 방향을 명확히 잡고, 30~100명 규모에 맞춰 불필요한 기능과 인프라를 의도적으로 배제했습니다. 그 결과 **별도 서버 없이 무료 플랜만으로 실서비스가 가능한 구조**를 확보했습니다.

1일

기획 → 배포 완료

3,109

코드 라인 (47개 파일)

18/18

MVP 요구 기능 검증

0원

월 운영 비용

해결한 문제

- 강의자료·발표자료가 개인 메일과 USB에 흩어져 재할용 불가
- 초청 회원만 접근해야 하는데 공유 수단이 없음
- 관리자 1명이 별도 교육 없이 운영할 수 있어야 함
- 예산 없이 시작해야 함

도출한 해법

- 공개 회원가입을 없애고 관리자 발급 방식으로 전환
- 이메일 대신 ID/PW 로그인 — 담당자 진입 장벽 최소화
- 모든 자료를 단일 콘텐츠 모델로 통합해 운영 부담 제거
- 서버리스 구성으로 인프라 관리·비용을 0으로

이 프로젝트의 차별점

AI 코딩 에이전트(Claude Code)를 실무 도구로 활용해, 기존에 팀 단위로 나누던 기획·설계·구현·테스트·배포·장애 대응을 **1인 워크플로우로 재구성**했습니다. 단순히 코드를 생성시킨 것이 아니라, **요구사항 판단·아키텍처 결정·보안 검증·원인 분석**을 직접 주도하고 에이전트를 구현 수단으로 운용했습니다. 상세 과정은 13페이지에 정리했습니다.

IN OPERATION
2026년 8월 14일 배포 완료 · 정상 운영 중

초청 회원 전용 비공개 서비스이므로 접속 주소와 계정 정보는 본 문서에서 제외했습니다. 실물 확인이 필요한 경우 별도 안내드립니다.

프로젝트 개요

프로젝트	HRD Strategic Circle — Private Knowledge Portal
배경	2027 HRD 전략 라운드테이블 참여 기업 HRD 담당자를 대상으로 한 초청형 지식 네트워크 운영 필요
목표	리더십 · AI 교육설계 · 기업교육 사례 · 인사이트 · 라운드테이블 자료를 회원 전용으로 아카이빙하고 열람·다운로드 제공
사용자	초기 30~50명, 최대 100명 내외의 중견기업 이상 HRD 담당자 / 관리자 1명
기간	2026년 8월 14일 (1일) — 요구사항 분석부터 프로덕션 배포 및 검증까지
역할	단독 수행 — 요구사항 분석 · 정보구조 설계 · UI/UX 디자인 · 프론트엔드 · 백엔드 · DB 설계 · 보안 설계 · 배포 · QA · 장애 대응
산출물	운영 중인 웹 서비스, 소스 코드(47개 파일 / 3,109라인), DB 스키마 SQL, 운영 문서(README)

핵심 UX 방향 설정

가장 먼저 결정한 것은 **무엇을 만들지가 아니라 무엇을 만들지 않을지**였습니다. HRD 담당자 네트워크라는 말에서 흔히 떠올리는 커뮤니티 구조를 배제하고, 자료 접근 동선에 집중했습니다.

배제한 구조

사람 → 게시판 → 커뮤니티

댓글 · 좋아요 · 팔로우 · 프로필 등 상호작용 기능은 운영 부담만 늘리고 자료 접근을 방해한다고 판단

채택한 구조

사람 → 지식 → 자료 → 오프라인 네트워크

온라인은 자료 아카이브 역할에 집중하고, 관계 형성은 오프라인 라운드테이블이 담당하도록 역할 분리

의도적으로 만들지 않은 기능

아래 항목은 구현 능력의 문제가 아니라, 30~100명 규모의 초청형 서비스에 필요하지 않다고 판단해 제외했습니다. 기능을 덜어낸 만큼 관리자 1명이 감당할 수 있는 운영 구조가 되었습니다.

공개 회원가입 · 이메일 인증 · OAuth(Google/Kakao/Naver) · 사용자 프로필 · 댓글 · 게시판 · DM · 좋아요 · 북마크 · 팔로우 · 포인트 · 랭킹 · 푸시/이메일/SMS 알림 · 결제 · 구독 · 자체 영상 호스팅 · 실시간 기능 · AI 챗봇 · 추천 알고리즘 · Analytics 대시보드 · LMS 진도율 · 시험 · 퀴즈 · 수료증

시스템 아키텍처

별도 백엔드 서버 없이 **서버리스 2계층**으로 구성했습니다. Netlify가 Next.js 서버 코드를 실행하고, Supabase가 데이터·인증·파일을 담당합니다. AWS · Docker · Nginx · Redis · 자체 CMS 없이 운영되며, 인프라 관리 비용이 발생하지 않습니다.

Browser

Desktop · Laptop · Tablet · Mobile — 반응형 웹 단일 코드베이스

Netlify — Next.js Server

middleware 라우트 보호 · 서버 컴포넌트 렌더링 · 관리자 API(회원 발급/초기화/삭제)
Storage 서명 URL 발급 · Service Role Key 실행 영역

Supabase

PostgreSQL (profiles · contents · files) · Auth (비밀번호 해시 · 세션)
Storage (private bucket) · Row Level Security

권한 검사를 두 계층에 이중으로 배치

화면에서 메뉴를 숨기는 방식은 권한 제어가 아니라고 보고, **서버 코드**와 **데이터베이스** 양쪽에서 독립적으로 검증하도록 설계했습니다. 애플리케이션에 결함이 생기더라도 DB 정책이 최종 방어선으로 남습니다.

1차 — 서버

- middleware가 비로그인 요청을 `/login` 으로 차단
- 관리자 페이지가 서버에서 role 재확인
- 관리자 API 전용 가드 함수 통과 필수

2차 — 데이터베이스

- 3개 테이블 전체 RLS 활성화
- Storage 객체 접근 정책 별도 적용
- 정책 재귀 방지를 위한 SECURITY DEFINER 함수 분리

검증 결과

배포된 실제 주소에서 비로그인 상태로 직접 요청해 차단 여부를 확인했습니다.

비로그인 상태 요청	결과
GET <code>/contents</code> — 자료 목록	로그인 화면으로 차단
GET <code>/admin</code> — 관리자 화면	로그인 화면으로 차단
GET <code>/contents/[id]</code> — 자료 상세	로그인 화면으로 차단
HRD STRATEGIC CIRCLE POST <code>/api/admin/members</code> — 회원 발급 API 직접 호출	로그인 화면으로 차단
GET <code>/api/files/[id]/download</code> — 파일 API 직접 호출	로그인 화면으로 차단

기술 스택과 선정 근거

영역	선택	선정 근거
Framework	Next.js 15 (App Router)	서버 컴포넌트로 권한 검사와 데이터 조회를 서버에서 처리 가능. middleware로 라우트 보호를 한 곳에 집중시킬 수 있어 보안 로직 분산을 방지
Language	TypeScript (strict)	DB 스키마와 화면 사이 타입 불일치를 컴파일 시점에 차단. <code>any</code> 사용 최소화
UI	Tailwind CSS v4	디자인 토큰(Navy/Gold)을 CSS 변수로 정의하고 전 화면에 일관 적용. 별도 UI 라이브러리를 도입하지 않아 번들 크기와 학습 비용 절감
Font	Pretendard	한글 가독성이 우수한 무료 웹폰트. 라이선스 비용 없음
Database	Supabase (PostgreSQL)	RLS를 통해 DB 레벨 권한 제어가 가능. 별도 백엔드 서버 없이 안전한 데이터 접근 구현
Auth	Supabase Auth	비밀번호 해시 저장·세션 관리를 검증된 구현에 위임. 직접 구현 시 발생할 보안 위험 회피
Storage	Supabase Storage	비공개 버킷 + 단기 서명 URL로 파일 유출 경로 차단
Video	YouTube Embed	자체 영상 호스팅 시 발생하는 트래픽 비용과 인코딩 부담 제거. 비공개 자료는 미등록(Unlisted) 영상 활용
Hosting	Netlify	Next.js 서버 실행을 공식 지원하며 무료 플랜에서 상업적 사용이 허용됨 . 대안 (Vercel Hobby)은 약관상 상업적 사용이 제한되어 기관 서비스에 부적합하다고 판단

의존성 최소화 원칙

라이브러리를 추가할 때마다 유지보수 대상과 보안 취약점 노출면이 함께 늘어난다고 보고, 직접 구현이 합리적인 경우를 구분했습니다.

도입하지 않은 것

- **WYSIWYG 에디터** — 관리자 1명이 쓰는 본문 입력에 무거운 에디터는 과잉. 줄바꿈을 보존하는 단순 입력으로 대체
- **Markdown 렌더러** — 라이브러리 1개와 XSS 방어 로직이 함께 필요해지므로 보류
- **UI 컴포넌트 라이브러리** — 필요한 컴포넌트가 9종뿐이라 직접 작성
- **스켈레톤 UI 라이브러리** — 단순 로딩 상태로 충분

직접 구현한 것

- **YouTube URL 정규화** — `youtu.be` · `watch?v=` · `shorts` · `embed` 4종 형식 자동 변환
- **카테고리 아이콘 5종** — 초청장 디자인에 맞춘 라인 아이콘을 SVG로 작성
- **브로셔 슬라이드** — 가로 스크롤 스냅 + 자동 전환 + 키보드/휠/터치 대응
- **점검·404 안내 화면** — 브랜드 톤에 맞춘 일러스트를 벡터로 제작

최종 런타임 의존성은 `next`, `react`, `react-dom`, `@supabase/supabase-js`, `@supabase/ssr` 5개입니다.

데이터 모델과 보안 설계

테이블 3개로 끝낸 데이터 모델

자료 유형별로 테이블을 나누면 유형이 늘어날 때마다 화면·API·쿼리가 함께 늘어납니다. 라운드테이블 자료까지 **단일 콘텐츠 모델**로 통합하고 카테고리 값으로만 구분해, 카테고리 추가 시 코드 한 줄만 수정하면 되도록 만들었습니다.

PROFILES

id · login_id · name
company · position · role
created_at

인증 정보와 분리

비밀번호는 Auth가 해시로 보관,
여기엔 표시용 정보와 권한만 저장

CONTENTS

id · title · category
body · youtube_url
is_published · created_at ·
updated_at

단일 통합 모델

별도 events 테이블 없이 카테고리로
라운드테이블 자료까지 수용

FILES

id · content_id
file_name · storage_path
created_at

공개 URL 미보관

비공개 버킷이라 영구 URL이 존재할 수
없어, 경로만 저장하고 요청 시점에
발급

적용한 보안 조치

항목	구현 방식
비밀번호 평문 저장 금지	Supabase Auth가 해시로만 보관. 발급된 비밀번호는 응답에서 1회만 전달하고 어디에도 기록하지 않음
Service Role Key 노출 차단	해당 클라이언트 파일에 <code>server-only</code> 를 선언해, 실수로 브라우저 코드에 포함되면 빌드가 실패하도록 강제
관리자 API 보호	모든 관리자 API가 공통 가드 함수를 먼저 통과. 서버에서 role을 직접 조회해 검증
파일 접근 제어	비공개 버킷 + 로그인 확인 후 60초 유효 서명 URL 발급. 링크가 유출되어도 곧 만료
SQL Injection	쿼리 빌더가 값을 파라미터로 전달. 검색어의 패턴 문자는 별도 필터링
한글 파일명 처리	스토리지 키는 무작위 UUID로 생성하고 원본 파일명은 DB에만 저장 — 인코딩 문제와 경로 추측을 동시에 차단
오류 메시지	사용자에게는 정돈된 안내만 노출. 내부 오류와 스택 트레이스는 표시하지 않음
비밀 정보 관리	<code>.env.local</code> 을 <code>.gitignore</code> 에 등록하고, 저장소에는 값이 비어 있는 예시 파일만 포함

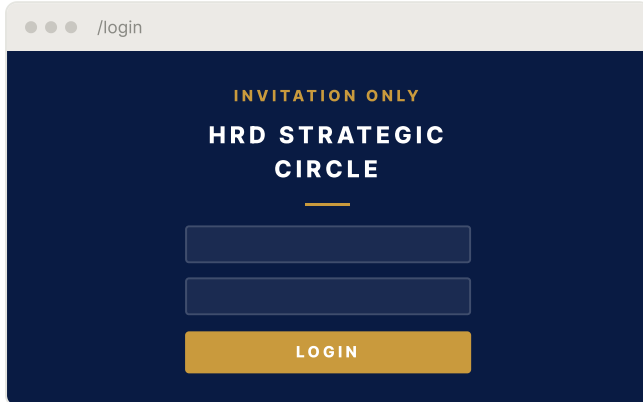
설계 판단 — 삭제된 컬럼 이야기

초기 요구사항에는 `files.file_url` 컬럼이 있었습니다. 그러나 버킷을 비공개로 설정하면 영구 URL이 존재할 수 없어 이 컬럼은 항상 비어 있게 됩니다. **요구사항을 그대로 구현하는 대신 근거를 밝히고 컬럼을 제외했으며**, 다운로드 요청 시점에 서명 URL을 발급하는 방식으로 대체했습니다. 의미 없는 컬럼을 남기면 이후 개발자가 잘못 사용할 여지가 생긴다고 판단했습니다.

주요 화면 (1)

행사 초청장 인쇄물을 시각 레퍼런스로 삼아 **Navy #091B43** · **Gold #C99A3D** 를 축으로 하는 디자인 시스템을 정의했습니다. Gold는 구분선 · 강조 버튼 · 활성 상태에만 제한적으로 사용해 과하지 않은 인상을 유지했습니다.

로그인 — Invitation Only



ID / PASSWORD 두 필드만 노출

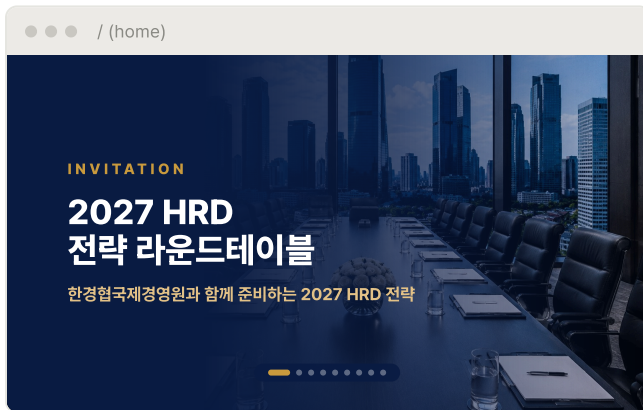
이메일을 숨긴 로그인

인증 시스템은 이메일을 요구하지만, HRD 담당자에게 이메일 관리 부담을 지우지 않기 위해 화면에는 **ID와 비밀번호만** 노출했습니다.

사용자 입력 : 발급 ID
↓ (서버 내부 변환)
인증 시스템 : 발급 ID + 내부 도메인

사용자는 내부 규칙을 알 필요가 없습니다. 관리자는 문자로 ID와 비밀번호만 전달하면 되고, 회원은 그대로 입력하면 로그인됩니다.

홈 — 9장 구성 브로셔



표지 → 소개 → 카테고리 4장 → 프로그램 → 행사정보 → 자료실

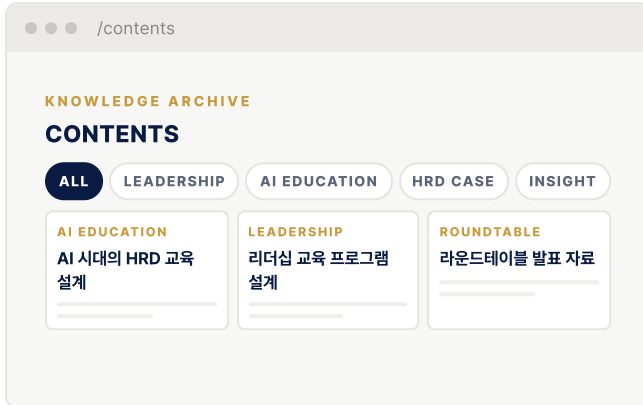
로그인 없이 볼 수 있는 공개 랜딩

초기에는 로그인 화면이 첫 관문이었으나, 처음 방문한 담당자에게 아무 정보도 주지 못한다는 문제가 있었습니다. **홈을 공개 브로셔로 전환**하고 자료실만 인증 뒤에 두는 구조로 재설계했습니다.

- 가로 스크롤 스냅 + **5초 자동 전환**, 일시정지 제공
- 마우스 휠 · 키보드 방향키 · 모바일 스와이프 모두 대응
- 글자 크기가 화면 가로폭과 **세로 높이 양쪽**에 반응하도록 처리해 노트북 환경에서도 잘림 없음
- 프로그램 시간표 · 일시 · 장소 · 참가비를 한눈에 배치

주요 화면 (2)

자료실 — 카테고리 필터와 검색



카테고리 5종 필터 · 제목 검색 · 카드형 목록

규모에 맞는 검색 설계

초기 자료가 수십 건 규모이므로 전문 검색 엔진은 과잉이라고 판단했습니다. **카테고리 필터 + 제목 검색**만 제공하고, 필터 상태를 주소에 담아 특정 카테고리 링크를 그대로 공유할 수 있게 했습니다.

상세 화면의 조건부 구성

- 영상이 등록되지 않은 자료는 **영상 영역 자체를 표시하지 않음**
- 첨부파일이 없으면 첨부 영역 미표시
- 자료가 없을 때도 화면이 깨지지 않도록 빈 상태 안내 제공

관리자 — 한 화면에서 끝나는 운영



Contents / Members 탭 · 계정정보 복사 (예시 데이터, 비밀번호는 마스킹)

관리자의 실제 업무 흐름에 맞춘 설계

관리자가 하는 일은 **이름과 회사를 입력하는 것**뿐입니다. ID는 `hrd001` 부터 자동 증가하고, 비밀번호는 혼동하기 쉬운 문자 (`0 / O` , `1 / l`)를 제외한 8자리로 생성됩니다.

발급 후 **[계정정보 복사]** 버튼을 누르면 문자로 바로 붙여넣을 수 있는 형태가 클립보드에 담깁니다. 사이트가 직접 메일이나 문자를 발송하지 않으므로 발송 실패·스팸 분류 같은 장애 요소가 없습니다.

삭제와 비밀번호 초기화에는 확인 절차를 두어 한 번의 실수로 실행되지 않도록 했습니다.

문제 해결 사례 (1)

실제 개발·배포 과정에서 발생한 장애와 해결 과정입니다. 모두 당일 안에 원인을 규명하고 조치했습니다.

01 배포 환경에서만 회원 발급 실패

- 증상** 로컬에서는 정상 동작하나, 배포된 사이트에서만 "회원 생성에 실패했습니다" 발생
- 1차 확인** 환경변수 등록 여부, 키 유효성, DB 권한을 각각 직접 호출해 검증 — 모두 정상. 저장된 키의 길이(219자)까지 원본과 일치
- 전환점** 세 가지 실패 지점이 동일한 메시지를 반환하도록 만든 것이 원인 파악을 막고 있었음을 인지. **실패 지점별로 다른 메시지를 반환하도록 개선**하고 재배포
- 원인** 호스팅 환경변수에 저장된 값이 실제 키가 아니라 **화면에서 가려진 상태로 표시되던 문자열**(`eyJhbGciOi...`)이었음. 마스킹이 원래 길이를 유지해 길이 검증까지 통과. 유니코드 문자 `•` (U+2022)는 HTTP 헤더에 담을 수 없어 요청 자체가 생성되지 않았음
- 조치** 올바른 키로 재등록 후 재배포. 관리자 전용 API에 한해 실패 원인을 함께 반환하도록 유지
- 배운 점** 오류 메시지를 뭉뚱그리면 운영 단계에서 원인 규명 비용이 급증한다. **사용자에게는 정돈된 안내를, 운영자에게는 구체적인 원인을** 제공하는 분리 설계가 필요하다

02 신규 인증 키 방식의 호환성 문제

- 증상** 플랫폼이 새로 발급한 secret 키로 모든 API 요청이 401 반환
- 분석** 인증 헤더 조합 4종(apikey 단독 / Bearer 단독 / 동시 / 인증 엔드포인트)을 각각 시험. 같은 방식에서 publishable 키는 정상 응답하는 것을 확인해 **코드가 아닌 키 자체의 문제로** 범위를 좁힘
- 조치** 레거시 방식 키로 전환해 즉시 정상화. 두 형식 모두 동작하도록 코드는 수정하지 않고 문서에 대응 관계를 명시
- 배운 점** 외부 서비스 문제와 자체 코드 문제를 **가장 낮은 계층에서 직접 호출해** 분리하면 불필요한 코드 수정을 피할 수 있다

03 특정 페이지에서만 화면 전환이 막힘

- 증상** 브로셔 9장 중 4장에서만 마우스 휠로 다음 장 전환이 되지 않음
- 가설과 반증** 내용이 길어 세로 스크롤이 생긴 것으로 추정하고 여백·글자 크기를 축소했으나 변화 없음. 측정해 보니 **네 페이지 모두 정확히 48px씩** 넘침 — 내용 길이가 원인이라면 값이 제각각이어야 하므로 가설 폐기
- 원인** 배경 장식 그래픽을 `-48px` 위치에 배치해 페이지 밖으로 내보냈는데, 이 영역이 스크롤 가능 범위로 계산되고 있었음. 해당 그래픽이 없는 나머지 페이지는 정상이었음
- 조치** 그래픽을 잘라내는 래퍼로 감싸 스크롤 범위에서 제외. 더불어 글자 크기가 세로 높이에도 반응하도록 개선해 낮은 해상도에서도 잘림이 없도록 처리
- 배운 점** **모든 페이지에서 오차가 동일하다는 사실**이 가장 강력한 단서였다. 증상을 정량적으로 측정하면 가설을 빠르게 기각할 수 있다

문제 해결 사례 (2)

04 개발 서버가 반복적으로 다운

- 증상** 작업 중 개발 서버가 `Internal Server Error` 를 반환하며 중단. 캐시 삭제 후 재시작하면 복구되지만 재발
- 원인** 번들러가 빌드 산출물 파일을 쓰는 도중 읽기가 겹치면서 발생하는 Windows 환경 파일 잠금 문제
- 조치** 개발 서버를 **Turbopack**으로 전환. 산출물 기록 방식이 달라 문제가 재발하지 않았고, 빌드 속도도 개선(약 15초 → 6초)

05 배포가 개발 서버를 중단시킴

- 증상** 배포를 실행할 때마다 로컬 개발 서버가 함께 죽음
- 원인** 배포 명령이 프로덕션 빌드를 수행하면서 개발 서버가 사용 중인 동일한 산출물 디렉터리를 덮어씀
- 조치** 배포 절차를 **개발 서버 중단 → 빌드·배포 → 개발 서버 재기동** 순으로 표준화. 이후 재발 없음
- 배운 점** 도구 간 자원 충돌은 코드가 아니라 **작업 순서**로 해결해야 하는 경우가 있다

06 관리자 계정 생성 규칙 불일치

- 증상** 관리자 계정을 만들었으나 로그인에 되지 않고 권한도 부여되지 않음
- 원인** 인증 시스템에 실제 이메일 주소로 계정을 생성했으나, 서비스는 `ID → 내부 이메일` 변환 규칙으로 동작하므로 매칭되는 계정이 없었음
- 조치** 계정 이메일을 내부 규칙에 맞게 변경하고 권한 정보를 등록. 이후 **초기 관리자 생성 절차를 운영 문서에 단계별로 명문화**
- 배운 점** 자동화되지 않는 최초 1회 설정일수록 **문서화의 가치가 크다**. 이 절차를 문서로 남기지 않았다면 인수인계 시점에 동일한 문제가 재현될 수 있었다

문제 해결에서 일관되게 적용한 방식

- **범위 좁히기** — 로컬/배포, 코드/외부서비스, 앱/DB로 구간을 나눠 어디까지 정상인지 먼저 확정
- **정량 측정** — 추측 대신 실제 값을 측정. "48px씩 동일"이라는 수치가 원인을 특정
- **관측 가능성 확보** — 원인이 안 보이면 먼저 보이게 만드는 작업(오류 세분화)에 투자
- **재발 방지** — 조치 후 절차·문서까지 갱신해 같은 문제가 반복되지 않도록 마감

AI Agent 협업 개발 방식

이 프로젝트는 AI 코딩 에이전트(Claude Code)를 구현 수단으로 활용해 1인 체제로 수행했습니다. 핵심은 **코드를 대신 작성하게 하는 것이 아니라, 사람이 판단해야 할 영역과 위임해도 되는 영역을 명확히 나눈 것이었습니다.**

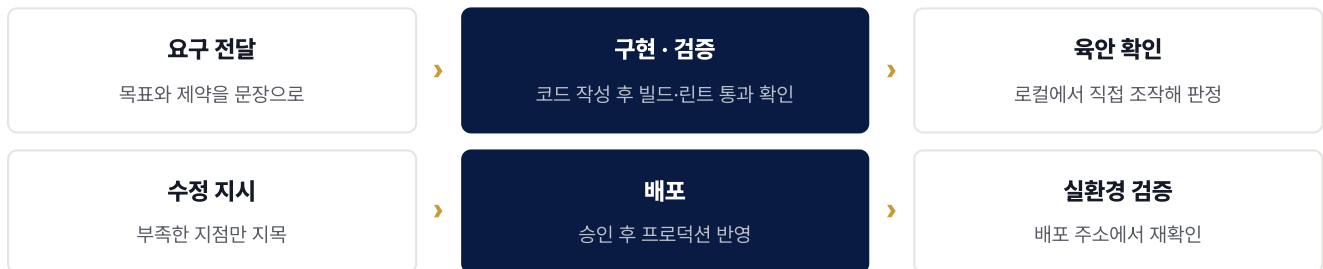
사람이 담당한 영역

- 요구사항 정의와 우선순위 결정
- 제품 방향 전환 판단 (로그인 우선 → 공개 브로셔)
- 호스팅·비용·약관 검토 및 최종 선택
- 디자인 방향과 완성도 판정
- 실제 화면에서의 동작 검증
- 운영 정책 결정 (공개 범위, 배포 시점)

에이전트에 위임한 영역

- 코드 작성 및 리팩터링
- DB 스키마·RLS 정책 SQL 작성
- 반복적인 스타일·반응형 처리
- 빌드·린트·타입 검사 실행
- 원인 후보 나열과 검증 스크립트 실행
- 문서 작성 및 배포 명령 수행

실제 작업 사이클



효과가 있었던 운용 방식

방식	구체적 효과
배포 권한을 분리	코드 수정은 자유롭게 맡기되 외부에 공개되는 배포는 승인 후 실행 하도록 규칙을 정해, 작업 중간 상태가 노출되는 것을 방지
검증을 사람이 수행	빌드 성공과 실제 동작은 다르다는 전제로, 모든 기능을 직접 눌러 확인. 배포 환경에서만 발생한 결함을 이 과정에서 발견
판단 근거를 요구	구현 방식이 요구사항과 다를 경우 이유를 명시하도록 하여, 임의 축소·확대를 통제 (예: file_url 컬럼 제외 근거)
단계별 보고	Phase 단위로 진행률과 완료 항목을 확인해, 작업이 요구사항에서 벗어나지 않도록 통제

한계와 대응

에이전트는 **실제 화면을 보고 판단하는 일과 외부 서비스의 실제 상태**를 알지 못했습니다. 환경변수에 잘못된 값이 저장된 사례, 디자인 완성도 판정, 자동 전환 속도 조정은 모두 사람의 확인이 있어야 해결됐습니다. 따라서 **구현 속도는 크게 높이지 검증 책임은 사람이 유지하는** 방식으로 운용했으며, 이 경계를 지킨 것이 하루 만에 실서비스 배포까지 도달한 요인이라고 판단합니다.

성과와 결론

MVP 요구 기능 18개 — 전 항목 검증 완료

기능	검증	기능	검증
ID/PW 비공개 로그인	완료	다중 파일 첨부	완료
관리자 계정 운영	완료	파일 다운로드	완료
관리자 기반 회원 발급	완료	콘텐츠 등록·수정·삭제	완료
홈 화면	완료	파일 업로드·삭제	완료
자료 목록	완료	회원 관리	완료
카테고리 필터	완료	비밀번호 초기화	완료
자료 상세	완료	계정정보 복사	완료
YouTube 영상 임베드	완료	로그아웃	완료
반응형 웹 (4개 환경)	완료	프로덕션 배포	완료

요구사항 외 추가 구현 : 공개 브로셔 랜딩 · 자동 전환 슬라이드 · 점검/404 안내 화면 · 관리자용 오류 진단 기능

정량 성과

1일 기획부터 배포까지	5개 런타임 의존성	3개 DB 테이블	0원 월 운영 비용
------------------------	----------------------	---------------------	----------------------

이 프로젝트에서 확인한 것

요구사항을 그대로 구현하는 것이 최선은 아니다

비공개 버킷과 양립할 수 없는 컬럼을 제외하고, 라이브러리 도입 대신 단순 입력 방식을 택하는 등 **근거를 밝히고 다르게 구현한 판단**이 결과적으로 코드와 운영 부담을 줄였습니다. 요구사항의 문구가 아니라 의도를 구현해야 한다는 점을 확인했습니다.

보안은 한 겹으로 충분하지 않다

화면에서 메뉴를 감추는 것은 권한 제어가 아니며, 서버 검사만으로도 부족합니다. **서버와 데이터베이스 양쪽에 독립적인 방어선**을 두었기에, 배포 후 API를 직접 호출하는 방식으로 검증했을 때도 모두 차단되는 것을 확인할 수 있었습니다.

진단 가능성은 기능만큼 중요하다

가장 오래 걸린 장애는 코드가 어려워서가 아니라 **원인이 보이지 않아서**였습니다. 실패 지점별로 다른 메시지를 반환하도록 바꾼 직후 원인이 특정됐습니다. 운영을 전제로 한 시스템에는 관측 가능성이 기능과 동등한 우선순위를 가져야 합니다.